

## Data Frame - Pandas

Pandas DataFrame is two-dimensional size-mutable, potentially heterogeneous tabular data structure with labeled axes (rows and columns). A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns. Pandas DataFrame consists of three principal components, the data, rows, and columns.

The diagram illustrates a Pandas DataFrame with 7 rows and 5 columns. The columns are labeled 'Name', 'Team', 'Number', 'Position', and 'Age'. The rows are indexed from 0 to 6. A blue arrow labeled 'Columns' points to the column headers. An orange arrow labeled 'Rows' points to the row indices. A purple box labeled 'Data' highlights a specific cell (Jonas Jerebko, 8.0) and its corresponding row and column labels. The table data is as follows:

|   | <i>Name</i>     | <i>Team</i>    | <i>Number</i> | <i>Position</i> | <i>Age</i> |
|---|-----------------|----------------|---------------|-----------------|------------|
| 0 | Avery Bradley   | Boston Celtics | 0.0           | PG              | 25.0       |
| 1 | John Holland    | Boston Celtics | 30.0          | SG              | 27.0       |
| 2 | Jonas Jerebko   | Boston Celtics | 8.0           | PF              | 29.0       |
| 3 | Jordan Mickey   | Boston Celtics | NaN           | PF              | 21.0       |
| 4 | Terry Rozier    | Boston Celtics | 12.0          | PG              | 22.0       |
| 5 | Jared Sullinger | Boston Celtics | 7.0           | C               | NaN        |
| 6 | Evan Turner     | Boston Celtics | 11.0          | SG              | 27.0       |

## Creating Data Frames

In the real world, a Pandas DataFrame will be created by loading the datasets from existing storage, storage can be SQL Database, CSV file, and Excel file. Pandas DataFrame can be created from the lists, dictionary, and from a list of dictionary etc. Dataframe can be created in different ways here are some ways by which we create a dataframe:1

### Creating a Data Frame from List

```
import pandas as pd
import numpy as np
lst = ['Geeks', 'For', 'Geeks', 'is', 'a', 'portal', 'for', 'geeks']
print(type(pd.DataFrame(lst)))
pd.DataFrame(lst)
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
0
0  Geeks
```

```
1 For
2 Geeks
3 is
4 a
5 portal
6 for
7 geeks
```

```
lst = [['Luv',25],['Lalita',53],['Nehal',28]]
pd.DataFrame(lst)
```

|   | 0      | 1  |
|---|--------|----|
| 0 | Luv    | 25 |
| 1 | Lalita | 53 |
| 2 | Nehal  | 28 |

Creating a data frame out of a dictionary

```
data = {
    'Name':['Luv','Lalita','Ratna','Nehal','Himanshu'],
    'Age':[25,53,25,28,34]
}
pd.DataFrame(data) #Will Create column names with the Keys we
have exclusively defined in the python dictionary
```

|   | Name     | Age |
|---|----------|-----|
| 0 | Luv      | 25  |
| 1 | Lalita   | 53  |
| 2 | Ratna    | 25  |
| 3 | Nehal    | 28  |
| 4 | Himanshu | 34  |

```
data = {
    'Name':['Luv','Lalita','Ratna','Nehal','Himanshu'],
    'Age':[25,53,25,28,34],
    'Address':
['Gurugram','Haldwani','Ghaziabad','Nainital','Nainital'],
    'Qualification': ['Engineer', 'M.Ed', 'Engineer', 'Doctor',
'Doctor']}
}
```

```
df = pd.DataFrame(data)
print(df)
print('-'*55)
print(df[['Name']]) #How to Select an individual column and it's data
from a data frame
```

|   | Name   | Age | Address   | Qualification |
|---|--------|-----|-----------|---------------|
| 0 | Luv    | 25  | Gurugram  | Engineer      |
| 1 | Lalita | 53  | Haldwani  | M.Ed          |
| 2 | Ratna  | 25  | Ghaziabad | Engineer      |

|   |          |    |          |        |
|---|----------|----|----------|--------|
| 3 | Nehal    | 28 | Nainital | Doctor |
| 4 | Himanshu | 34 | Nainital | Doctor |

```

-----
      Name
0      Luv
1    Lalita
2     Ratna
3     Nehal
4  Himanshu

```

## Slicing in DataFrames using Iloc and Loc

```

data = {'one'   : pd.Series([1, 2, 3, 4]),
        'two'   : pd.Series([10, 20, 30, 40]),
        'three' : pd.Series([100, 200, 300, 400]),
        'four'  : pd.Series([1000, 2000, 3000, 4000])}

```

```

df = pd.DataFrame(data)
df

```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 1   | 10  | 100   | 1000 |
| 1 | 2   | 20  | 200   | 2000 |
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

Python loc() function - The loc() function is label based data selecting method which means that we have to pass the name of the row or column which we want to select. This method includes the last element of the range passed in it, unlike iloc(). loc() can accept the boolean data unlike iloc(). Many operations can be performed using the loc() method like

```

print(df.loc[1])    #Returns the data from the 1st Row for all the
                    #subsequent columns
print('-'*55)
print(df.loc[2:])   #Returns the data starting fromt the second row
                    #till the end for all the subsequent columns
print('-'*55)
print(df.loc[1:,'three':'four']) #The Items before the comma gives the
                                #rows from where to where and after the comma decides the column from
                                #where to where. Note - When we use them together, they give a combined
                                #effect and not an individual effect
print('-'*55)
print(df.loc[:, 'two']) #Returns all the rows for column 'two'

```

```

one      2
two     20
three   200
four   2000
Name: 1, dtype: int64

```

```

-----
   one  two  three  four
2    3   30   300  3000
3    4   40   400  4000
-----

```

```

-----
   three  four
1    200  2000
2    300  3000
3    400  4000
-----

```

```

-----
0    10
1    20
2    30
3    40
-----

```

Name: two, dtype: int64

## Basic iloc Operations

The `iloc()` function is an indexed-based selecting method which means that we have to pass an integer index in the method to select a specific row/column. This method does not include the last element of the range passed in it unlike `loc()`. `iloc()` does not accept the boolean data unlike `loc()`.

df

```

   one  two  three  four
0    1   10   100  1000
1    2   20   200  2000
2    3   30   300  3000
3    4   40   400  4000

```

`df.iloc[:]` *#Returns the complete dataset, all the rows and all the columns*

`df.iloc[:,:]` *#Also returns all the rows and all the columns*

`df.iloc[:2,:]` *#First two rows and all the columns*

*#Note - The part before the column represents the data from the rows where to where we have to take them, The part after the comma represents the Columns from where to where we have to take them*

`df.iloc[1:-1, 1:-1]` *#Return data from 2nd row till second last row and second column to second last column*

```

   two  three
1    20   200
2    30   300

```

NOTE - The only difference between `loc` and `iloc` is that in `loc` we can pass the index/column/row names but in the `iloc` we can only pass the numbers/integers

`df.iloc[:,2]` *#Returns the data from only one specific column for all the subsequent rows*

```
df.iloc[:,2:3] #Returns the data from only one specific column for all the subsequent rows
df.iloc[[2,3],[0,1]] #Returns the data from the given indexes of Rows and Columns, We can explicitly decide the data for each row and column
```

|   | one | two |
|---|-----|-----|
| 2 | 3   | 30  |
| 3 | 4   | 40  |

Slicing using the conditions

```
df
```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 1   | 10  | 100   | 1000 |
| 1 | 2   | 20  | 200   | 2000 |
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

```
df['two'] #Returns the data from that particular Column
```

|   |    |
|---|----|
| 0 | 10 |
| 1 | 20 |
| 2 | 30 |
| 3 | 40 |

```
Name: two, dtype: int64
```

```
print([df['two']>20]) #Return true for the that satisfies the given condition
```

```
df[df['two']>20] #This will convert the given data into a dataframe and with the values that satisfies the condition and not the boolean values
```

|   |       |
|---|-------|
| 0 | False |
| 1 | False |
| 2 | True  |
| 3 | True  |

```
Name: two, dtype: bool]
```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

```
df.loc[df['two'] > 20, ['three','four']] #here we are giving to conditions for the pandas to return us the rows with condition given before the comma and columns given after the comma
```

|   | three | four |
|---|-------|------|
| 2 | 300   | 3000 |
| 3 | 400   | 4000 |

```
df.loc[df['three'] <300, ['one','four']]
```

|   | one | four |
|---|-----|------|
| 0 | 1   | 1000 |
| 1 | 2   | 2000 |

## Column addition in Data Frames

```
l = [22,33,44,55] #Created a new list
df['five'] = l #Added the new list in the dataframe.
df
```

|   | one | two | three | four | five |
|---|-----|-----|-------|------|------|
| 0 | 1   | 10  | 100   | 1000 | 22   |
| 1 | 2   | 20  | 200   | 2000 | 33   |
| 2 | 3   | 30  | 300   | 3000 | 44   |
| 3 | 4   | 40  | 400   | 4000 | 55   |

```
l = [100,565,2454,665]
df['six'] = pd.Series(l) #We can also use Pandas series for the same
df
```

|   | one | two | three | four | five | six  |
|---|-----|-----|-------|------|------|------|
| 0 | 1   | 10  | 100   | 1000 | 22   | 100  |
| 1 | 2   | 20  | 200   | 2000 | 33   | 565  |
| 2 | 3   | 30  | 300   | 3000 | 44   | 2454 |
| 3 | 4   | 40  | 400   | 4000 | 55   | 665  |

```
df['seven'] = np.array([555,222,555,222]) #We can also use numpy array
for the same
df
```

|   | one | two | three | four | five | six  | seven |
|---|-----|-----|-------|------|------|------|-------|
| 0 | 1   | 10  | 100   | 1000 | 22   | 100  | 555   |
| 1 | 2   | 20  | 200   | 2000 | 33   | 565  | 222   |
| 2 | 3   | 30  | 300   | 3000 | 44   | 2454 | 555   |
| 3 | 4   | 40  | 400   | 4000 | 55   | 665  | 222   |

## Using and manipulating an existing column

```
df['seven'] = df['seven'] + 5 #Change the column seven with the
contents of column seven increased by 5
df
```

|   | one | two | three | four | five | six  | seven |
|---|-----|-----|-------|------|------|------|-------|
| 0 | 1   | 10  | 100   | 1000 | 22   | 100  | 560   |
| 1 | 2   | 20  | 200   | 2000 | 33   | 565  | 227   |
| 2 | 3   | 30  | 300   | 3000 | 44   | 2454 | 560   |
| 3 | 4   | 40  | 400   | 4000 | 55   | 665  | 227   |

## Deleting a row/column from the Dataframe

```
del df['six']    #Deletes the given column  
df
```

|   | one | two | three | four | five | seven |
|---|-----|-----|-------|------|------|-------|
| 0 | 1   | 10  | 100   | 1000 | 22   | 560   |
| 1 | 2   | 20  | 200   | 2000 | 33   | 227   |
| 2 | 3   | 30  | 300   | 3000 | 44   | 560   |
| 3 | 4   | 40  | 400   | 4000 | 55   | 227   |

```
df.pop('seven') #Also Deletes the given column  
df
```

|   | one | two | three | four | five |
|---|-----|-----|-------|------|------|
| 0 | 1   | 10  | 100   | 1000 | 22   |
| 1 | 2   | 20  | 200   | 2000 | 33   |
| 2 | 3   | 30  | 300   | 3000 | 44   |
| 3 | 4   | 40  | 400   | 4000 | 55   |

## Addition of new Rows in the Data Frame

```
df1 = pd.DataFrame([[1, 2], [3, 4]], columns = ['a', 'b'])  
df2 = pd.DataFrame([[5, 6], [7, 8]], columns = ['a', 'b'])  
df3 = pd.DataFrame([[9, 9], [9, 9]], columns = ['a', 'b'])
```

```
df2 =pd.concat([df1,df2])    #Adding all the contents of df1 to df 2  
and saving the new dataframe to df2 but the indexes will be the same  
and one index will have more than one value and also a new reo will be  
added by the name index
```

```
print(df2)
```

```
df3 = pd.concat([df1,df3]).reset_index(drop=True) #This will reset the  
index to normal integers and drop=True will completely remove the  
Index column that was created due to the addition of both the  
dataframes
```

```
df3
```

|   | a | b |
|---|---|---|
| 0 | 1 | 2 |
| 1 | 3 | 4 |
| 0 | 5 | 6 |
| 1 | 7 | 8 |

|   | a | b |
|---|---|---|
| 0 | 1 | 2 |
| 1 | 3 | 4 |
| 2 | 9 | 9 |
| 3 | 9 | 9 |

## Pandas drop function

```
df =pd.DataFrame( { 'one'    : pd.Series([1, 2, 3, 4]),
                    'two'    : pd.Series([10, 20, 30, 40]),
                    'three'  : pd.Series([100, 200, 300, 400]),
                    'four'   : pd.Series([1000, 2000, 3000, 4000])
                  })
```

```
print(df)
```

```
df.drop([0,1],axis=0, inplace=True) #This will completely remove the
Row 0 and row 1 and inplace = True means that the original df has been
changed as well
```

```
df
```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 1   | 10  | 100   | 1000 |
| 1 | 2   | 20  | 200   | 2000 |
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

To remove a column

```
df.drop(['one','three'], axis=1, inplace=True) #Removes the given
columns from the original Df
```

```
df
```

|   | two | four |
|---|-----|------|
| 2 | 30  | 3000 |
| 3 | 40  | 4000 |

```
df =pd.DataFrame( { 'one'    : pd.Series([1, 2, 3, 4]),
                    'two'    : pd.Series([10, 20, 30, 40]),
                    'three'  : pd.Series([100, 200, 300, 400]),
                    'four'   : pd.Series([1000, 2000, 3000, 4000])
                  })
```

```
df.transpose() #Rows becomes our columns and vice versa
```

|       | 0    | 1    | 2    | 3    |
|-------|------|------|------|------|
| one   | 1    | 2    | 3    | 4    |
| two   | 10   | 20   | 30   | 40   |
| three | 100  | 200  | 300  | 400  |
| four  | 1000 | 2000 | 3000 | 4000 |

## A set of more Dataframe Functionalities

```
# axes attribute
print(df)
print('-'*35)
print(df.axes) #Gives a python list of all the rows and columns
labels/indices
print('-'*35)
print(df.ndim) #Gives the number of dimensions the dataframe has
print('-'*35)
print(df.dtypes) #Gives data types in all the columns of the
dataframes
print('-'*35)
print(df.shape) #Gives the order of the dataframe
print('-'*35)
print(df.head) #Returns only the first five rows of the dataset
print('-'*35)
print(df.tail) #Returns the last five rows of the dataset
print('-'*35)
print(df.head()) #Returns the first two rows of the dataset
```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 1   | 10  | 100   | 1000 |
| 1 | 2   | 20  | 200   | 2000 |
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

```
-----
[RangeIndex(start=0, stop=4, step=1), Index(['one', 'two', 'three',
'four'], dtype='object')]
-----
2
-----
one      int64
two      int64
three    int64
four     int64
dtype: object
-----
(4, 4)
-----
<bound method NDFrame.head of      one  two  three  four
0     1   10   100  1000
1     2   20   200  2000
2     3   30   300  3000
3     4   40   400  4000>
-----
<bound method NDFrame.tail of      one  two  three  four
0     1   10   100  1000
1     2   20   200  2000
2     3   30   300  3000
```

```
3      4      40      400     4000>
```

```
-----  
      one  two  three  four  
0      1   10   100  1000  
1      2   20   200  2000  
2      3   30   300  3000  
3      4   40   400  4000
```

## Statistical and Mathematical Function

```
data = {'one' : pd.Series([1, 2, 3, 4]),  
        'two' : pd.Series([10, 20, 30, 40]),  
        'three' : pd.Series([100, 200, 300, 400]),  
        'four' : pd.Series([1000, 2000, 3000, 4000])}
```

```
df = pd.DataFrame(data)  
df
```

```
      one  two  three  four  
0      1   10   100  1000  
1      2   20   200  2000  
2      3   30   300  3000  
3      4   40   400  4000
```

```
print(df.sum()) #Sums up all the data column wise  
print(df.mean())  
print(df.median())
```

```
one      10  
two      100  
three    1000  
four     10000  
dtype: int64  
one      2.5  
two     25.0  
three   250.0  
four   2500.0  
dtype: float64  
one      2.5  
two     25.0  
three   250.0  
four   2500.0  
dtype: float64
```

Mode

```
#Mode
```

```
de = pd.DataFrame({'A': [1, 2, 2, 3, 4, 4, 4, 5], 'B': [10, 20, 20,
```

```
30, 40, 40, 50, 60]})  
print(de['A'].mode())
```

```
0      4  
Name: A, dtype: int64
```

Variance

```
print(df)  
df.var()
```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 1   | 10  | 100   | 1000 |
| 1 | 2   | 20  | 200   | 2000 |
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

```
one      1.666667e+00  
two      1.666667e+02  
three    1.666667e+04  
four     1.666667e+06  
dtype: float64
```

Min

```
df.min()
```

```
one      1  
two     10  
three   100  
four   1000  
dtype: int64
```

Max

```
df.max()
```

```
one      4  
two     40  
three   400  
four   4000  
dtype: int64
```

Standard Deviation

```
df.std()
```

```
one      1.290994  
two     12.909944  
three   129.099445  
four   1290.994449  
dtype: float64
```

## Describe function

```
df['five'] = ['A','B','C','D']
print(df)
df.describe()
```

|   | one | two | three | four | five |
|---|-----|-----|-------|------|------|
| 0 | 1   | 10  | 100   | 1000 | A    |
| 1 | 2   | 20  | 200   | 2000 | B    |
| 2 | 3   | 30  | 300   | 3000 | C    |
| 3 | 4   | 40  | 400   | 4000 | D    |

|       | one      | two       | three      | four        |
|-------|----------|-----------|------------|-------------|
| count | 4.000000 | 4.000000  | 4.000000   | 4.000000    |
| mean  | 2.500000 | 25.000000 | 250.000000 | 2500.000000 |
| std   | 1.290994 | 12.909944 | 129.099445 | 1290.994449 |
| min   | 1.000000 | 10.000000 | 100.000000 | 1000.000000 |
| 25%   | 1.750000 | 17.500000 | 175.000000 | 1750.000000 |
| 50%   | 2.500000 | 25.000000 | 250.000000 | 2500.000000 |
| 75%   | 3.250000 | 32.500000 | 325.000000 | 3250.000000 |
| max   | 4.000000 | 40.000000 | 400.000000 | 4000.000000 |

## Pipe Function

*#Pipe function helps us to apply a specific function to the whole data frame*

```
print(df)
df.drop(['five'], axis=1, inplace=True)
print(df)
```

```
def add_function(i,j) :
    return i+j
```

```
def sub_fuction(i,j) :
    return i-j
```

```
df.pipe(add_function, 10)
```

|   | one | two | three | four | five |
|---|-----|-----|-------|------|------|
| 0 | 1   | 10  | 100   | 1000 | A    |
| 1 | 2   | 20  | 200   | 2000 | B    |
| 2 | 3   | 30  | 300   | 3000 | C    |
| 3 | 4   | 40  | 400   | 4000 | D    |

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 1   | 10  | 100   | 1000 |
| 1 | 2   | 20  | 200   | 2000 |
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 11  | 20  | 110   | 1010 |

|   |    |    |     |      |
|---|----|----|-----|------|
| 1 | 12 | 30 | 210 | 2010 |
| 2 | 13 | 40 | 310 | 3010 |
| 3 | 14 | 50 | 410 | 4010 |

Multiple functions in a single line

```
df
```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 1   | 10  | 100   | 1000 |
| 1 | 2   | 20  | 200   | 2000 |
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

```
def mean_(col) :
    return col.mean() #Will apply the mean function column wise and
the col here is a variable not syntax
```

```
def square(i) :
    return i**2
```

```
check = df.pipe(mean_)
print(check, type(check))
```

```
df.pipe(mean_).pipe(square) #First it will apply the mean function to
all the columns and will return a pandas series then it will apply the
square function to all the items of the new pandas series
```

```
df = pd.Series([1,5,4,5,4])
print(type(df))
df.pipe(square)
```

```
one          2.5
two          25.0
three        250.0
four        2500.0
dtype: float64 <class 'pandas.core.series.Series'>
<class 'pandas.core.series.Series'>
```

```
0      1
1     25
2     16
3     25
4     16
dtype: int64
```

Apply function

```
df = pd.DataFrame(data)
df
```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 1   | 10  | 100   | 1000 |
| 1 | 2   | 20  | 200   | 2000 |
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

```
print(df.apply(np.mean))    #Apply is used to apply a function to
                             specific columns or rows of a dataframe or the entire dataframe
print(df.apply(np.max))
```

```
one          2.5
two          25.0
three        250.0
four         2500.0
dtype: float64
one           4
two           40
three         400
four          4000
dtype: int64
```

Using apply with Lambda functions

```
df.apply(lambda x:x.max() - x.min())
```

```
one          3
two          30
three        300
four         3000
dtype: int64
```

## Map function

The map() method in a Pandas DataFrame allows you to apply a function to each element of a specific column of the DataFrame. The function can be either a built-in Python function or a user-defined function.

df

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 1   | 10  | 100   | 1000 |
| 1 | 2   | 20  | 200   | 2000 |
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

```
df.map(lambda x : x*100)
```

|   | one | two  | three | four   |
|---|-----|------|-------|--------|
| 0 | 100 | 1000 | 10000 | 100000 |
| 1 | 200 | 2000 | 20000 | 200000 |

```

2  300  3000  30000  300000
3  400  4000  40000  400000

df = pd.DataFrame({
    'A' : [1.2,2.3,2.1,5.5],
    'B' : [7.8,9.9,10.2,1.1]
})
print(df)

newDf = df.map(lambda x : x+1)
newDf

```

|   | A   | B    |
|---|-----|------|
| 0 | 1.2 | 7.8  |
| 1 | 2.3 | 9.9  |
| 2 | 2.1 | 10.2 |
| 3 | 5.5 | 1.1  |

|   | A   | B    |
|---|-----|------|
| 0 | 2.2 | 8.8  |
| 1 | 3.3 | 10.9 |
| 2 | 3.1 | 11.2 |
| 3 | 6.5 | 2.1  |

## Reindex Function

The reindex function in Pandas is used to change the row labels and/or column labels of a DataFrame. This function can be used to align data from multiple DataFrames or to update the labels based on new data. The function takes in a list or an array of new labels as its first argument and, optionally, a fill value to replace any missing values. The reindexing can be done along either the row axis (0) or the column axis (1). The reindexed DataFrame is returned.

```

df = pd.DataFrame(data)
df

```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 1   | 10  | 100   | 1000 |
| 1 | 2   | 20  | 200   | 2000 |
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

## Reindexing Rows

```

df.reindex([2,1,3,0])  #We can shuffle the rows in any order we like
using reindex

```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 2 | 3   | 30  | 300   | 3000 |
| 1 | 2   | 20  | 200   | 2000 |

|   |   |    |     |      |
|---|---|----|-----|------|
| 3 | 4 | 40 | 400 | 4000 |
| 0 | 1 | 10 | 100 | 1000 |

## Reindexing Columns

```
df.reindex(columns=['two', 'three', 'four', 'one'])
```

|   | two | three | four | one |
|---|-----|-------|------|-----|
| 0 | 10  | 100   | 1000 | 1   |
| 1 | 20  | 200   | 2000 | 2   |
| 2 | 30  | 300   | 3000 | 3   |
| 3 | 40  | 400   | 4000 | 4   |

*#To perform both the operations together*

```
print(df)
df.reindex([2,1,3,0],columns=['two', 'three', 'four', 'one'])
#Performing both the operations together
```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 1   | 10  | 100   | 1000 |
| 1 | 2   | 20  | 200   | 2000 |
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

|   | two | three | four | one |
|---|-----|-------|------|-----|
| 2 | 30  | 300   | 3000 | 3   |
| 1 | 20  | 200   | 2000 | 2   |
| 3 | 40  | 400   | 4000 | 4   |
| 0 | 10  | 100   | 1000 | 1   |

## Renaming the columns of the Data Frame

```
df
```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 1   | 10  | 100   | 1000 |
| 1 | 2   | 20  | 200   | 2000 |
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

```
dfCopy = df.copy()
dfCopy.rename(columns={"one": "newOne", "two": "newTwo"},
inplace=True) #Renaming the columns
dfCopy
```

|   | newOne | newTwo | three | four |
|---|--------|--------|-------|------|
| 0 | 1      | 10     | 100   | 1000 |
| 1 | 2      | 20     | 200   | 2000 |
| 2 | 3      | 30     | 300   | 3000 |
| 3 | 4      | 40     | 400   | 4000 |

```
dfCopy.rename(index={0:"zero"},inplace=True)
dfCopy
```

|      | newOne | newTwo | three | four |
|------|--------|--------|-------|------|
| zero | 1      | 10     | 100   | 1000 |
| 1    | 2      | 20     | 200   | 2000 |
| 2    | 3      | 30     | 300   | 3000 |
| 3    | 4      | 40     | 400   | 4000 |

```
df
```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 1   | 10  | 100   | 1000 |
| 1 | 2   | 20  | 200   | 2000 |
| 2 | 3   | 30  | 300   | 3000 |
| 3 | 4   | 40  | 400   | 4000 |

## Sorting in Pandas Data Frame

```
unsortedDF = pd.DataFrame({
    "one": pd.Series([2, 1, 39, 14]),
    "two": pd.Series([10, 20, 30, 40]),
    "three": pd.Series([100, 200, 300, 400]),
    "four": pd.Series([1000, 2000, 3000, 4000]),
})
```

```
unsortedDF
```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 0 | 2   | 10  | 100   | 1000 |
| 1 | 1   | 20  | 200   | 2000 |
| 2 | 39  | 30  | 300   | 3000 |
| 3 | 14  | 40  | 400   | 4000 |

*#Sorting a column data-*

*# It will also sort the indexes with it. and the corresponding columns will also be changed.*

```
unsortedDF.sort_values(by="one")
```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 1 | 1   | 20  | 200   | 2000 |
| 0 | 2   | 10  | 100   | 1000 |
| 3 | 14  | 40  | 400   | 4000 |
| 2 | 39  | 30  | 300   | 3000 |

*#Sorting ascending or descending*

```
unsortedDF.sort_values(by="two", ascending=False) #This will sort in descending manner
```

*#It will also sort the indexes with it. and the corresponding columns will also be changed.*

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 3 | 14  | 40  | 400   | 4000 |
| 2 | 39  | 30  | 300   | 3000 |
| 1 | 1   | 20  | 200   | 2000 |
| 0 | 2   | 10  | 100   | 1000 |

*# Sort in specific order based on multiple columns*

```
unsortedDF.sort_values(by = ['one', 'four'])
```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 1 | 1   | 20  | 200   | 2000 |
| 0 | 2   | 10  | 100   | 1000 |
| 3 | 14  | 40  | 400   | 4000 |
| 2 | 39  | 30  | 300   | 3000 |

Sort the values by specific sorting algorithm

- Quick Sort
- Merge Sort
- Heap Sort

```
unsortedDF.sort_values(by=['one'], kind='heapsort')
```

|   | one | two | three | four |
|---|-----|-----|-------|------|
| 1 | 1   | 20  | 200   | 2000 |
| 0 | 2   | 10  | 100   | 1000 |
| 3 | 14  | 40  | 400   | 4000 |
| 2 | 39  | 30  | 300   | 3000 |

### ### Group By Function

```
cricket = pd.DataFrame(  
    {  
        'Team'      : ['India', 'India', 'Australia',  
        'Australia', 'SA', 'SA', 'SA', 'SA', 'NZ', 'NZ', 'NZ', 'India'],  
        'Rank'      : [2, 3, 1, 2, 3, 4, 1, 1, 2, 4, 1, 2],  
        'Year'      :  
        [2014, 2015, 2014, 2015, 2014, 2015, 2016, 2017, 2016, 2014, 2015, 2017],  
        'Points'    :  
        [876, 801, 891, 815, 776, 784, 834, 824, 758, 691, 883, 782]  
    }  
)
```

cricket

|   | Team  | Rank | Year | Points |
|---|-------|------|------|--------|
| 0 | India | 2    | 2014 | 876    |
| 1 | India | 3    | 2015 | 801    |

|    |           |   |      |     |
|----|-----------|---|------|-----|
| 2  | Australia | 1 | 2014 | 891 |
| 3  | Australia | 2 | 2015 | 815 |
| 4  | SA        | 3 | 2014 | 776 |
| 5  | SA        | 4 | 2015 | 784 |
| 6  | SA        | 1 | 2016 | 834 |
| 7  | SA        | 1 | 2017 | 824 |
| 8  | NZ        | 2 | 2016 | 758 |
| 9  | NZ        | 4 | 2014 | 691 |
| 10 | NZ        | 1 | 2015 | 883 |
| 11 | India     | 2 | 2017 | 782 |

```
cric = cricket.groupby('Team') #Creates a pandas inbuilt dictionary
with keys as Team names and values as an array of indexes where the
item is found
```

```
type(dict(cric.groups))
```

```
dict
```

```
cricket.groupby('Year').groups
```

```
{2014: [0, 2, 4, 9], 2015: [1, 3, 5, 10], 2016: [6, 8], 2017: [7, 11]}
```

```
cricket.groupby(['Team', 'Year']).get_group(('Australia', 2014))
```

|   | Team      | Rank | Year | Points |
|---|-----------|------|------|--------|
| 2 | Australia | 1    | 2014 | 891    |